Assignment 3

Path Integral Control

Overview

Path integral control generalizes KL control from linearly solvable MDPs (Chapter 7) to continuous state spaces. The free-energy representation of the value function (Chapter 6) allows the optimal control to be approximated by sampling K forward trajectories and computing an importance-weighted average. This assignment covers both the discrete-time and continuous-time formulations.

The discrete-time formulation follows from the variational principle (Chapter 6) applied to deterministic dynamics $x_{t+1} = F(x_t, u_t)$ with noise injected into the control signal. In continuous time, the Cole-Hopf transform $Z = \exp(-V/\lambda)$ converts the HJB equation into a PDE for the desirability function (Section 8.4). Under the **matching condition** (Theorem 8.1)

$$\Sigma \;=\; \lambda\, G\, R^{-1}\, G^{\top}, \tag{1}$$

where R is the control cost matrix, this PDE linearizes, and the Feynman-Kac formula expresses its solution as a path integral over the reference diffusion $dx = f(x,u)\,dt + G(x)\,dW$.

In this assignment you will implement both the discrete-time and continuous-time versions and test them on a double integrator, a nonholonomic unicycle, a cart-pole swing-up, and a 13D fixed-wing aircraft.

Implementation

Integrators (`mppi/integrators.py`)

1. Forward Euler

Implement `euler_step(dynamics_fn, x, u, dt, xp)`.

Discretize the ODE $\dot{x} = f(x,u)$ using a single forward Euler step. The discrete-time examples call this through their model's `step()` method.

2. Euler-Maruyama

Implement `euler_maruyama_step(drift_fn, diffusion_fn, x, u, dt, xp)`.

Discretize the SDE $dx = f(x,u)\,dt + G(x)\,dW$. The drift f and diffusion matrix G are provided as callables. The noise dimension is determined by the shape of G.

Solver (`mppi/core.py`)

3. Importance weights (Eq. 8.13)

Implement `_compute_weights(self, costs)`.

Given K trajectory costs, subtract the minimum cost for numerical stability, exponentiate the negative costs scaled by `self.lambda_`, and normalize to sum to 1. If the total weight is near zero, return uniform weights.

4. Discrete-time solve loop

Implement `solve(self, x0)`.

Roll out K trajectories over the horizon T using `self.model.step()`, accumulate running and terminal costs, compute importance weights via `_compute_weights()`, and update `self.U` as the weighted average of the perturbed control sequences. Boilerplate (state array initialization, state clamping) is provided.

5. Continuous-time solve loop

Implement `solve_continuous(self, x0)`.

Same structure as `solve()`, but after each `self.model.step()` call, inject process noise through the diffusion matrix $G(x)$ obtained via `self.model.diffusion()`. Sample Brownian increments ΔW of the correct variance and add $G\,\Delta W$ to the propagated state.

Experiments

After completing the implementation, run each system below. Use `--animate` for real-time visualization and `--env` to select different obstacle environments.

Double integrator

A 2D point mass with state $[x, y, \dot{x}, \dot{y}]$ and acceleration control $[a_x, a_y] \in [-5, 5]^2\,\mathrm{m/s^2}$. The running cost penalizes distance to goal, velocity, control effort, and obstacle penetration. Obstacles are soft Gaussian penalties loaded from the environment YAML.

```
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/forest.yaml --animate
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/u_trap.yaml --animate
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/three_mountains.yaml --animate
```

Unicycle

A nonholonomic vehicle with state $[x, y, \theta]$ and control $[v, \omega]$ (forward speed and yaw rate). The nonholonomic constraint prevents lateral motion, requiring anticipatory turning for obstacle avoidance.

```
python examples/discrete/unicycle.py --animate
python examples/discrete/unicycle.py \
    --env config/environments/forest.yaml --animate
python examples/discrete/unicycle.py \
    --env config/environments/u_trap.yaml \
    --T 200 --sigma 2.0 --animate
```

Cart-pole swing-up

An underactuated system with state $[x, \dot{x}, \theta, \dot{\theta}]$ and a single horizontal force $F \in [-10, 10]$ N applied to the cart. The pole starts hanging ($\theta = \pi$) and must be swung to upright ($\theta = 0$). The cost penalizes angle error from upright, cart displacement, and control effort. The swing-up is nonconvex: the controller must pump energy before stabilizing the pole at the inverted equilibrium.

```
python examples/discrete/cartpole.py --animate
```

Fixed-wing aircraft

A 13D 6DOF rigid body (position, body-frame velocity, quaternion, angular rates) with 4D control (aileron, elevator, throttle, rudder). Aerodynamic forces and moments are computed from angle of attack and sideslip. A nominal TECS controller provides the warm-start trajectory. This example uses the same `_compute_weights` and `solve` implementations from `mppi/core.py`.

```
python examples/discrete/fixed_wing/nominal.py
python examples/discrete/fixed_wing/mppi.py --K 128 --jit
python examples/discrete/fixed_wing/nominal.py --noise
python examples/discrete/fixed_wing/mppi.py --K 128 --jit --noise
```

With $K = 128$ on CPU the solver is too slow for real-time control and uses too few samples to reliably handle process noise. See the GPU acceleration section below for a practical configuration.

Analysis

1. **Horizon and covariance (forest).** Run the double integrator on `forest` with default parameters, then with increased horizon and covariance:

```
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/forest.yaml --animate
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/forest.yaml \
    --T 200 --sigma 5.0 --animate
```

Why do the default parameters produce only local obstacle avoidance? How does increasing T and σ together enable longer-range planning?

2. **Symmetry breaking (double slit).** Run the double integrator on `double_slit` with low and high covariance:

```
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/double_slit.yaml \
    --sigma 1.0 --animate
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/double_slit.yaml \
    --sigma 5.0 --animate
```

Why does low σ cause the controller to stall at the symmetric barrier? How does high σ break the symmetry?

3. **Horizon length (u-trap).** Run the double integrator on `u_trap` with short and long horizons:

```
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/u_trap.yaml --T 50 --animate
python examples/discrete/double_integrator_navigation.py \
    --env config/environments/u_trap.yaml --T 200 --animate
```

Why does the short horizon lead the controller into the trap? What is the approximate minimum T needed to plan around the barrier?

4. **Exploration noise (cart-pole).** Run the cart-pole with $\sigma \in \{0.5,\ 1.0,\ 3.0,\ 10.0\}$ (`--sigma <value>`). Why does the swing-up fail at both very low and very high σ? Relate this to the control bounds $F \in [-10, 10]\,\mathrm{N}$.

Optional: GPU acceleration

With a CUDA-capable GPU, the MPPI solver runs significantly faster at high sample counts:

```
python examples/discrete/fixed_wing/mppi.py --K 256
python examples/discrete/fixed_wing/mppi.py --K 256 --jit
python examples/discrete/fixed_wing/mppi.py --K 1024 --gpu
```

```
python examples/discrete/unicycle.py --K 1024
python examples/discrete/unicycle.py --K 1024 --gpu
python examples/discrete/unicycle.py --K 4096 --gpu
```

Why does the fixed-wing benefit much more from GPU acceleration than the unicycle?

With GPU acceleration and $K = 1024$ samples, the solver runs fast enough for real-time control and uses enough trajectories to keep the fixed-wing performant under process noise:

```
python examples/discrete/fixed_wing/mppi.py --K 1024 --gpu --noise
```

Optional: continuous-time SDE formulation

Compare the discrete-time and continuous-time solvers on the unicycle:

```
python examples/discrete/unicycle.py --animate
python examples/continuous/unicycle_sde.py --animate
```

How does the trajectory distribution differ when noise enters through the diffusion matrix $G(x)$ rather than through additive control perturbations?

Deliverables

- Completed `mppi/integrators.py` (`euler_step` and `euler_maruyama_step`).
- Completed `mppi/core.py` (`_compute_weights`, `solve`, and `solve_continuous`).
- Successful runs on all four systems (double integrator, unicycle, cart-pole, fixed-wing).
- Written responses to analysis questions 1–4.